

Assignment 1

Student: Francesco Gnan
Student's email: gnanf@usi.ch

Polynomial regression

In this assignment we want to perform polynomial regression of $p(z)$ using gradient descent with PyTorch. $p(z)$ is defined as

$$p(z) = \sum_{k=1}^4 z^k w_k = \frac{z^4}{100} - z^3 = z^2 - 10z, \quad (1)$$

where $\mathbf{z} \in \mathbb{R}$. This task is implemented in file `Francesco.Gnan.py`. In the following we discuss exercise by exercise the code we provided.

1. The solution to this question can be found between line 22 and 41. In particular, we defined two functions, `polynomial(z, coeffs)` and `plot_polynomial(coeffs, z_range, color='b')`. The former one is called within the latter one so as to plot the exact polynomial in Eq.1. We wrote these functions taking into account that the polynomial can be expressed as the dot product of two vectors: $p(z) = \mathbf{w}^T \mathbf{x}$, where $\mathbf{w} = [w_0, w_1, w_2, w_3, w_4]^T = [0, -10, 1, -1, 0.01]^T$ and $\mathbf{x}^i = [z_i^0, z_i, z_i^2, z_i^3, z_i^4]$. Note that when we defined the `np.array` of the weights, we included the constant term 0. The computed plot is shown in Figure1. We observe that the function is not linear inside the interval $(-3, 3)$.
2. Between lines 41 and 52 we defined a function `create_dataset(coeffs, z_range, sample_size, sigma, seed)` which is useful to create the training and the validation datasets. In detail, the function get as input the vector of the weights $\mathbf{w}$, it defines the vector $\mathbf{x}^i$ and then it compute $p(z)$ with `np.dot()`. Finally it adds a gaussian random noise to each $p(z)$ value with a certain standard deviation `sigma`.
3. In lines 52-64 we used the previous function in order to create two distinct (different random seed) datasets, one for training and the other one for evaluation. They have the same size equal to 500 samples and they share the same value of `sigma`. The training dataset will be used to train our model. Once the model is trained, we want to test it on new data (evaluation dataset), not seen by the model itself during the training. We follow this approach because we want to understand if or model is able to generalize well, i.e. if it works well also with new sets of data.
4. Between lines 64 and 84 we defined and applied the function `visualize_data(X, y, coeffs, z_range, title)` so as to create two plots: in the first one we compare the exact polynomial with a scatter plot of the training data, whereas in the second one, we compare it with evaluation data. They are shown in Figure 2. It appears that the noise is not high and there are no outliers, so we expect that training on this data will lead to good results, i.e. once trained, the model will be able to predict the polynomial within the region where such training occurred.
5. In this exercise we want to perform a polynomial regression for $p(z)$ using a linear regression model, i.e. we want the model to find the best estimation of the weights w_i . In Detail:

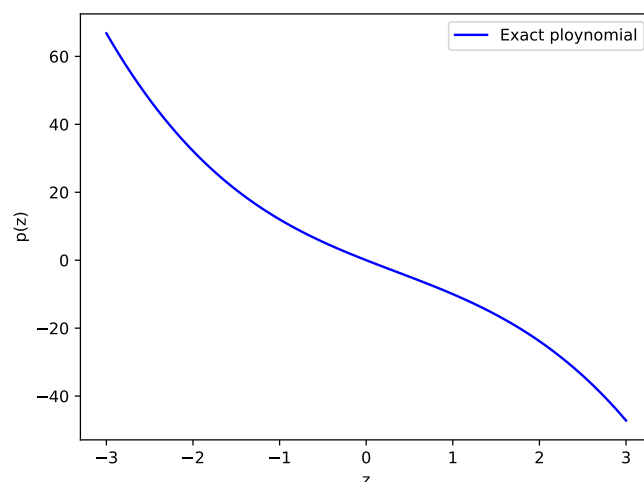


Figure 1: Ex.1, exact polynomial

- in lines 86-88 we let our machine decide to use the CPU or the GPU depending on their availability.
- in lines 90-93 we defined our linear model which takes 5 values as input, since we have 5 weights to estimate (taking into account w_0) and returns a one-dimensional output. We set the so-called Mean Squared Error as our loss function and we decided to use the Stochastic Gradient Descent as our optimizer. The latter has a learning rate of 0.001.
- from line 95 to line 109 we reshaped our data, converted them into PyTorch tensors and then we move everything to the desired and available device.
- from line 112 to line 150 we trained our model.
- within a for loop of length `n_steps = 5000` at each step we repeat the following actions:
 - firstly we use the training dataset
 - we set the gradient accumulator of our optimizer to 0;
 - we compute the output of the model and we use it to evaluate the training loss;
 - we compute the gradients and we use them to make our optimizer take a step in finding the best parameters that minimize the loss
 - secondly we use the evaluation data in order to compute the current output of our model
 - we finally use that evaluation output to compute the evaluation loss

At the end of the loop we expect that both the training and the evaluation loss converge to zero, i.e. that our model has learnt the true parameters of the function that generated our data.

We consider our training procedure good when the training loss is less than 0.5. We chose a learning rate equal to 0.001 because it allows us to reach the desired loss after about 3000 steps. If we set a too high learning rate value (for example 0.1), we would get very high loss values or nan values. This would happen because the learning rate decides length of each SGD step. If it is too long, than the algorithm would diverge and never find a minimum. On the other hand, if the learning rate is too low (for example 0.000001), finding the best loss minimum would require a very large number

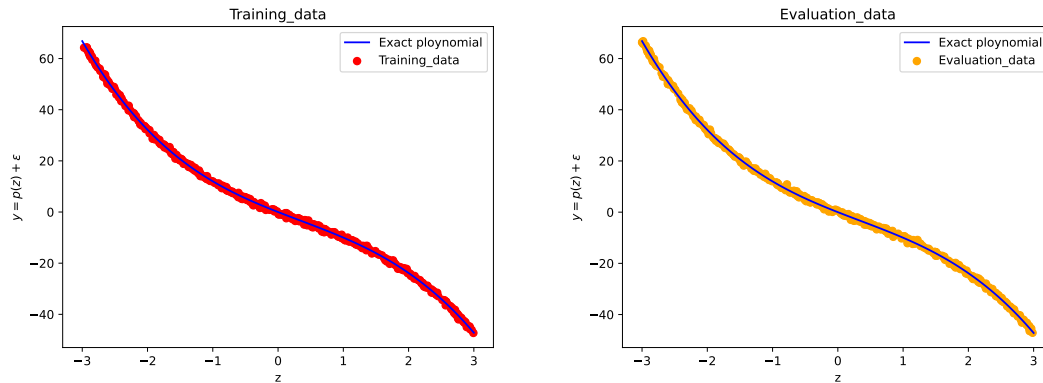


Figure 2: Ex.4 Training and evaluation datasets

of steps and thus a high computational cost. In `torch.nn.Linear` we set `bias` as `False` since we implemented a linear model with 5 input values and we are already taking into account w_0 . In this exercise we readjusted some code examples presented during the lecture on linear regression.

6. In order to graphically judge the performance of our training, from line 154 to line 169 we plot the training and evaluation loss as functions of the training steps. This plot is shown in Figure 3. We observe that both the training and the evaluation loss reach values that are close to the expected 0. Moreover, we present a zoom over a random region and we check that within that steps interval the training loss is greater than the evaluation one. We expected the training loss to be lower than the evaluation loss. The first one measures how well your model fits the training data, while the second one measures how well our model generalizes to unseen (different random seed) data. Anyway the two lines are very close and we consider our training as a success.
7. From line 173 to line 188 we used the coefficients estimated by our model in order to plot a prediction on new data within the training region. This result is shown in Figure 4. We observe a good prediction also outside the training region since we did not use activation functions and we have not introduced non-linearities. In other words, our one-layer network learns the right weights of the polynomial and then it is in principle able to represent the polynomial everywhere. If we expand the range at a certain point the match is no good anymore. This is due to the fact that the estimations are not perfect. If we train our model for a larger number of steps probably we can reach an estimation of the weights which is closer to the true values.
8. In order to judge the quality of our estimations, we plot the value of the parameters at each training step and we make a comparison with the true values. These plots are reported in Figure 5, where we can see that after 5000 steps there is a general convergence to the true values. We observe that only w_0 tends to fail in the prediction which is supposed to be zero. We also observe for w_3 and w_4 discontinuous jumps at the beginning of the training.
9. We wanted then to understand how the sample size of the training and evaluation data affects the training performances. To do so we re-trained our model using the same previous parameters, but changing the sample sizes to: `train_sample_size=10` and `eval_sample_size=500`. We repeated exactly the same procedure as before between lines 239 and 304. We plotted again the comparison between the training and the evaluation loss as shown in Figure 6. This graph suggests that we are observing overfitting: in fact, the training loss reaches zero almost immediately, while the evaluation

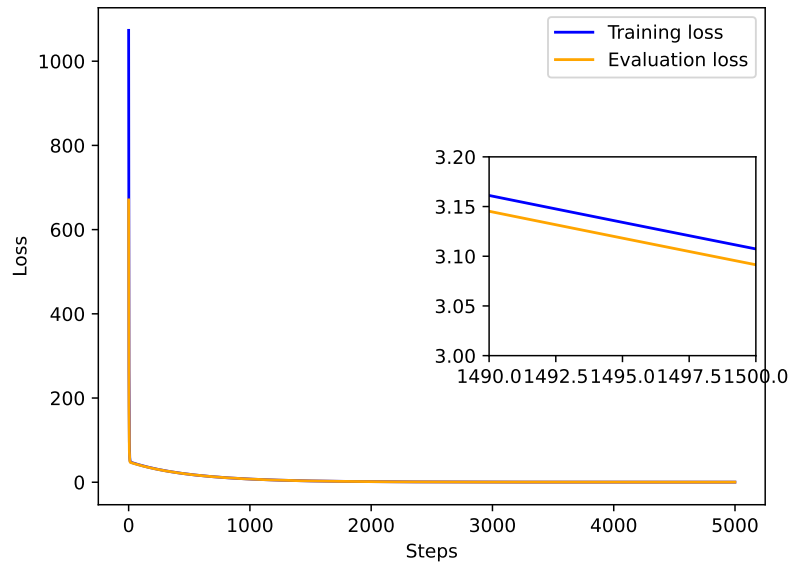


Figure 3

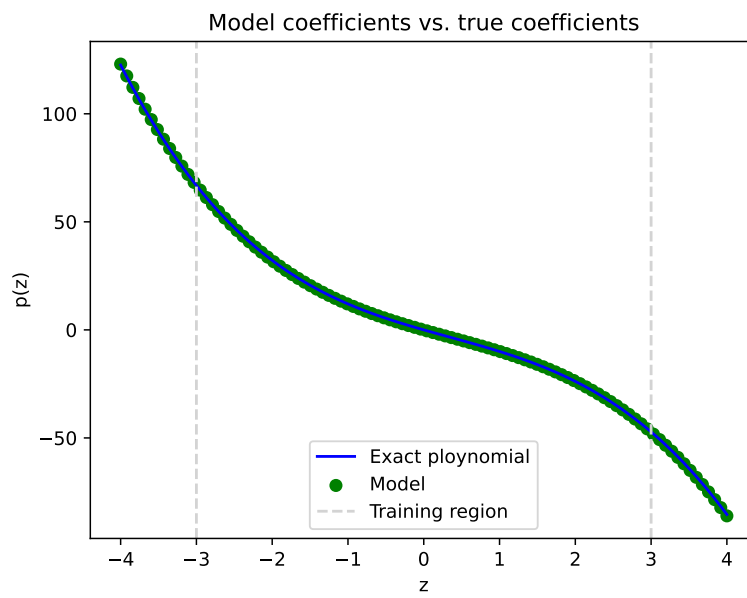
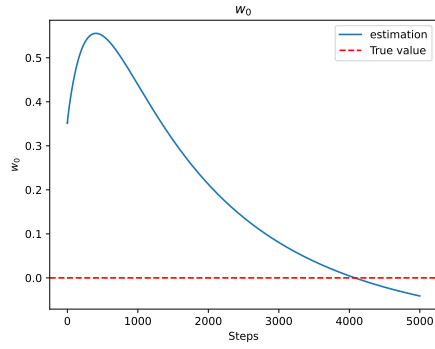
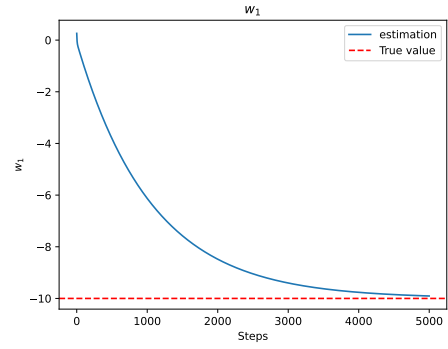


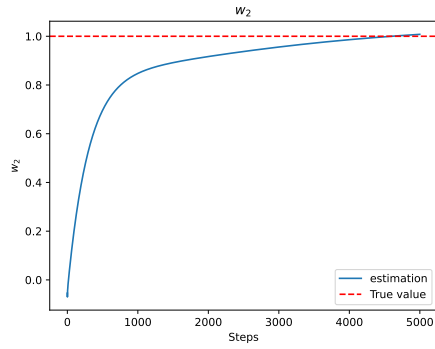
Figure 4



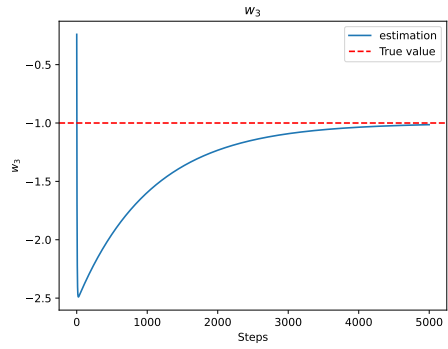
(a) w_0



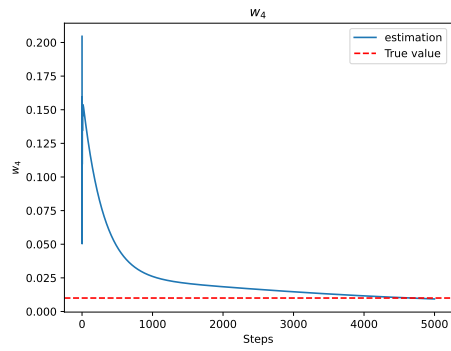
(b) w_1



(c) w_2



(d) w_3



(e) w_4

Figure 5

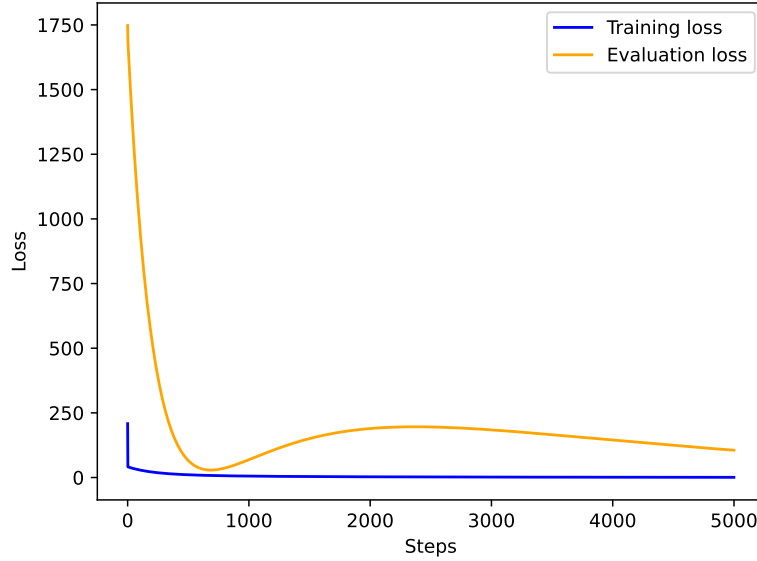


Figure 6

loss converges to ≈ 112 . This could mean that our model, having a small amount of training data, has learnt principally the noise from the training data without learning a general pattern behind the data. For this reason it is not able to generalize well on new unseen data (evaluation data). We have understood that the two datasets, training and evaluation, should have almost the same size. Moreover, it is preferable to have a larger training dataset: 10 values are too few for a good training, especially if the data are affected by some noise, as always happens in real-life problems.

In Figure 7 we tried to construct the polynomial within the training region using the estimated parameters and we compare the result with the exact function. We observe what we expected: the prediction is not as good as the one in Figure 3. We implemented it between lines 315 and 330.

10. We now want to learn the function

$$f(x) = 5 \sin x + 3 \quad (2)$$

we suppose our data x^i to belong to the interval $[a, a]$ and we compare to different cases: in the first one we perform a linear regression for $a = 0.01$; in the second one we consider a larger interval for $a = 5$. In the first case the function is almost linear and then we expect our linear model with one input and one output (with `bias=True` to take into account the intercept) to find good estimations for the parameters. In the other case instead we expect not satisfying results, since we try to learn a non-linear function with a linear model.

- $a = 0.01$: This exercise is implemented from line 346 to line 443. We reused and modified the code implemented for the previous exercises. Figure 8 shows the noisy data we generated according to the function in Eq.2.

In Figure 9 we report the training and evaluation loss for each training step. As expected they reach zero indicating a good performance of our model.

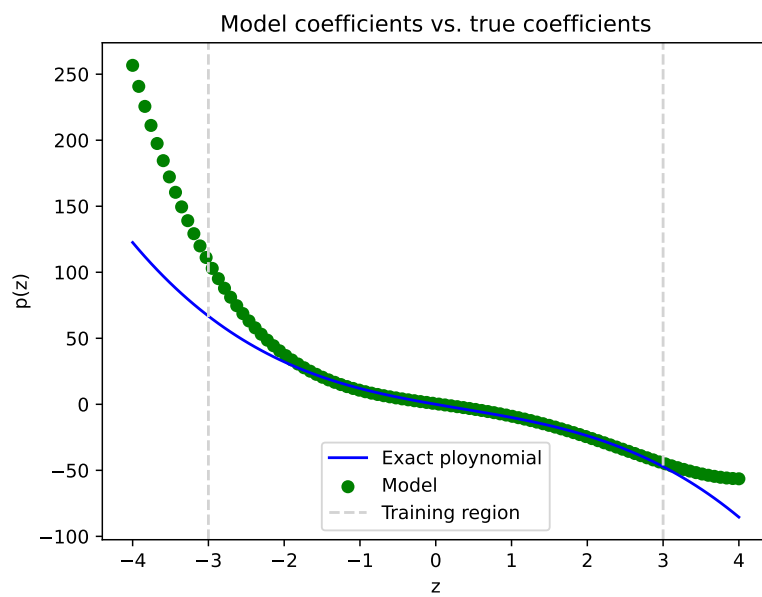


Figure 7

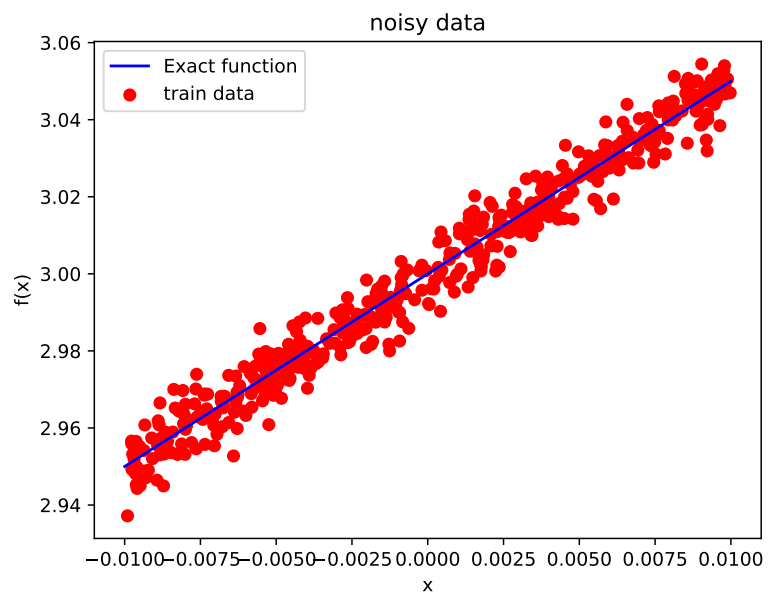


Figure 8

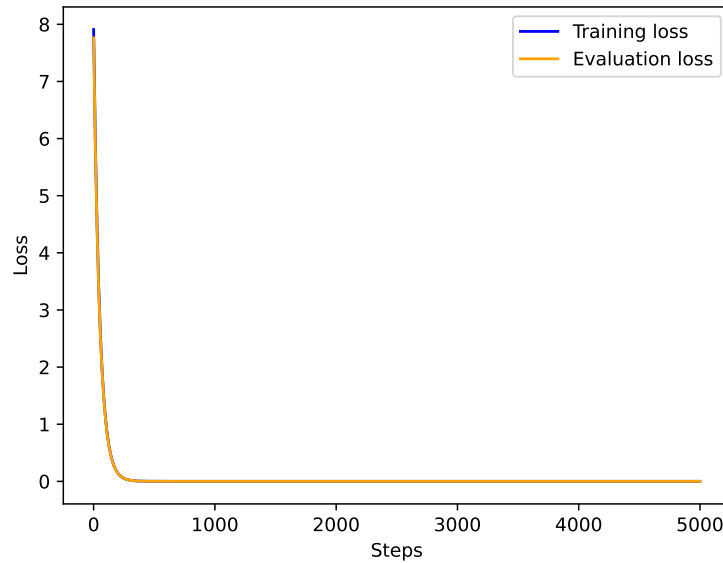


Figure 9

- $a = 5$: From line 445 until the end of the code we solve this exercise. Figure 10 shows the noisy data we generated, while Figure 11 reveals that, as we predicted, the linear model is not able to learn a non-linear function (the loss doesn't converge to zero). If we changed the number of inputs to n , our linear model would learn at least the Taylor expansion approximation of the target function up to the n^{th} order.

Questions

- We think that could be beneficial to choose two different values of α (learning rate) for Equation (3) and (4) even if at the beginning of our training we randomly initialize all the parameters. So we don't know where the research for the loss minimum starts. In particular, if we used different learning rates, it would be possible to better tune the hyper-parameters. A certain weight could have a greater impact to the construction of the target function compared to the others. So, with different learning rates we could discover which are the most relevant ones and focus on them during the training.
- Choosing two different learning rates in principle means that we fix the hyper-parameters values and they remain the same during all the training phase. Instead, adaptive methods modify the learning rates during the training taking into account the steepness of the region they are exploring, i.e. they increase or decrease α in a certain position taking into account the gradient value at that position in the parameters space. For example, "the AdaGrad algorithm, individually adapts the learning rates of all model parameters by scaling them inversely proportional to the square root of the sum of all the historical squared values of the gradient. The parameters with the largest partial derivative of the loss have a correspondingly rapid decrease in their learning rate, while parameters with small partial derivatives have a relatively small decrease in their learning rate. The net effect is greater progress in

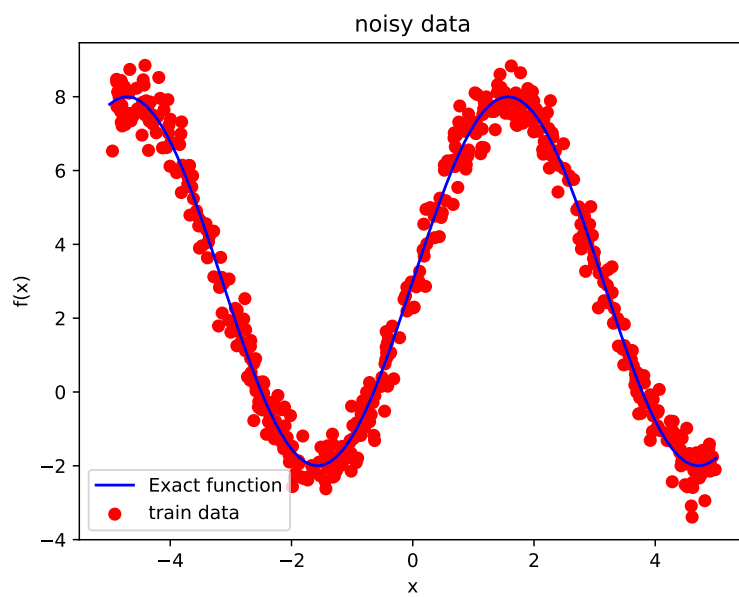


Figure 10

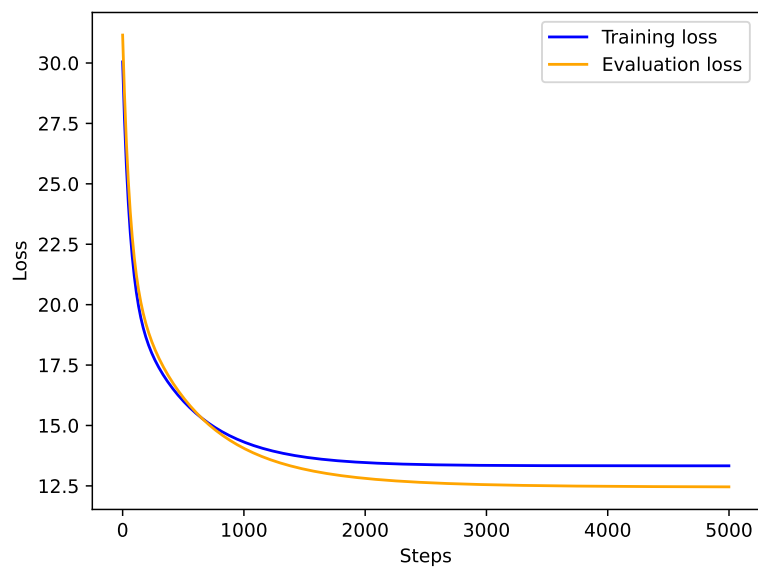


Figure 11

the more gently sloped directions of parameter space”¹.

¹Deep Learning Book